

AI-POWERED CRIMINAL NETWORK ANALYSIS SYSTEM

Technology Stack • System Architecture • Data Flow • Graph Analytics • AI/NLP • Database •
Frontend/Backend

Problem Statement ID	26189
Organization	Ministry of Home Affairs
Department	National Crime Records Bureau (NCRB), Women Safety Division
Prototype objective	Connect fragmented crime/intelligence data, uncover relationships, detect suspicious patterns and provide evidence-backed intelligence to investigators.

Core design principle: build one complete, explainable loop: raw fragmented data → entities → resolved entities → relationships → graph → suspicious pattern → evidence → investigator dashboard.

Source basis: SIH 2026 problem-statement material and the project MVP roadmap supplied in this conversation.

1. Executive Summary

The prototype is an investigator decision-support system. It ingests synthetic/fictitious FIR-style text, CDR data and financial transaction data; extracts entities; resolves duplicates; creates evidence-backed relationships; stores structured information in PostgreSQL; constructs a NetworkX graph; calculates network metrics; detects suspicious patterns; and presents the result through Streamlit and PyVis.

Recommended core stack

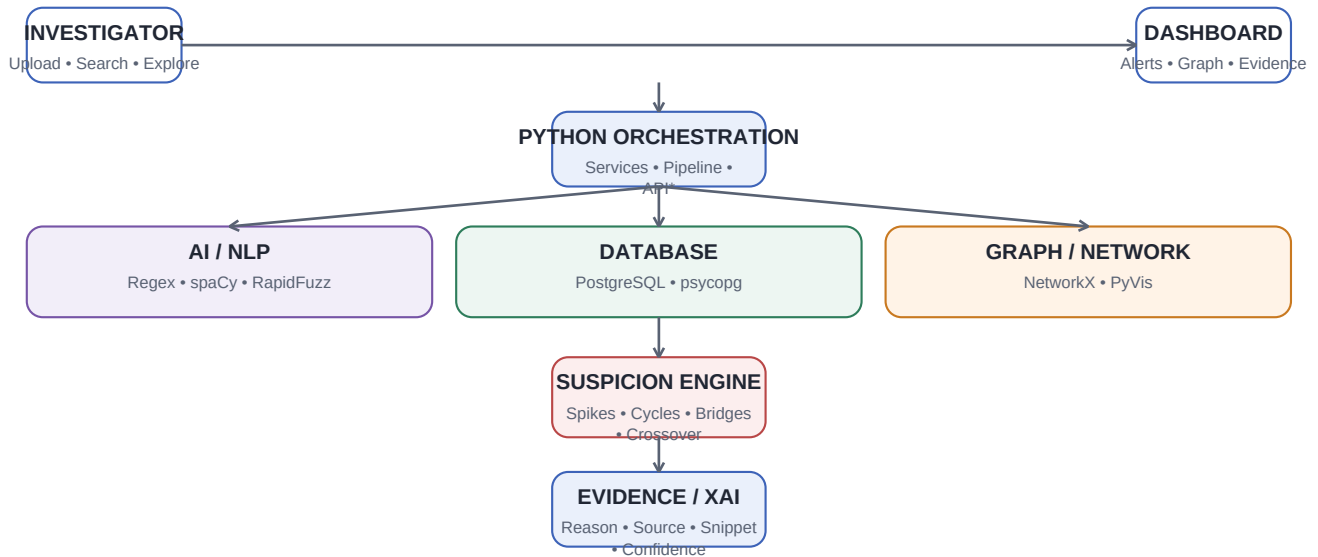
- **AI/NLP:** Python Regex + spaCy + RapidFuzz + Pandas.
- **Database:** PostgreSQL + psycopg.
- **Graph:** NetworkX + PyVis.
- **Detection:** deterministic rules for communication spikes, circular money flow, bridge entities and multi-source crossover.
- **Evidence/XAI:** source IDs, snippets/context, reason codes, confidence and graph metrics.
- **Frontend:** Streamlit investigator dashboard.
- **Backend:** Python orchestration; FastAPI is optional if a clean API boundary is useful.
- **ML:** scikit-learn only after the rule-based MVP works.

The demo must prove

- FIR text, CDR CSV and transaction CSV can be uploaded.
- Entities are extracted and at least some duplicates are resolved.
- Relationships retain source/context and confidence.
- PostgreSQL stores the structured source of truth.
- NetworkX reveals meaningful graph structure.
- Four suspicious-pattern rules generate explainable alerts.
- Each alert opens a WHY FLAGGED panel with evidence.
- The graph is interactive in the dashboard.
- Synthetic/fictitious data is used for the prototype and the system is presented as decision support, not guilt determination.

2. Overall System Architecture

End-to-End System Architecture



* FastAPI is optional for the time-constrained MVP.

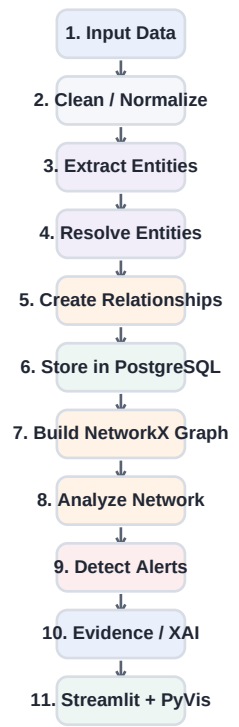
The system has a user-facing layer, a Python orchestration layer, three technical pillars and an evidence-backed detection layer. PostgreSQL is the persistent source of truth; NetworkX is the graph analysis layer; the AI/NLP components perform extraction and resolution; Streamlit/PyVis present the results.

Architecture rule

Do not make every technology responsible for every problem. Use the simplest tool that produces a testable result. In particular, use rules and graph analytics for the first suspicious-pattern engine instead of a black-box classifier.

3. End-to-End Data Flow

Prototype Data Flow



Raw data → entities → relationships → graph → alert → evidence → investigator.

Implementation sequence

Stage	Implementation
Input	FIR text, CDR CSV, financial transactions and supported records.
Preprocess	Normalize text, phones, dates, identifiers and CSV fields.
Extract	Regex handles structured patterns; spaCy handles natural-language entities.
Resolve	Normalize → exact match → fuzzy match → attribute overlap → confidence → merge/review.
Relate	Create typed relationships such as CONTACTED, TRANSFERRED, LOCATED_AT, USED and ASSOCIATED_WITH.
Store	Persist entities, documents, transactions, relationships, alerts and evidence in PostgreSQL.
Graph	Load stored relationships into a directed/weighted NetworkX graph.
Analyze	Calculate degree/betweenness, shortest paths, connected components and community structure.
Detect	Run communication-spike, circular-flow, bridge and multi-source rules.
Explain	Attach source IDs, snippets, confidence, reason codes and relevant metrics.
Present	Streamlit filters/search/alerts + PyVis interactive network.

4. PART A — AI / ML

The AI layer should focus on extraction and entity resolution. The MVP does not need a large custom neural model. Reliability, traceability and explainability are more valuable for the first prototype.

AI/NLP stack

Technology	Purpose	Priority	How to implement
Regex / re	Phone, vehicle, case ID, account-like IDs, dates, emails	P1	Build and test 5–8 project-specific patterns
spaCy	PERSON, LOCATION, ORGANIZATION and text entities	P1	Pretrained NER + EntityRuler/Matcher
Pandas	Clean/process CDR and transaction CSVs	P1	read_csv, filtering, groupby, merge, missing-data handling
RapidFuzz	Duplicate/similar entity matching	P1	Normalization + similarity + supporting attributes + confidence
scikit-learn	Optional learned anomaly layer	P3	Add only after deterministic rules work
LLM	Optional evidence-grounded case summary	P3	Summarize retrieved evidence; never invent findings

4.1 Regex extraction

Use predictable-format extraction for phone numbers, vehicle registrations, account-like IDs, case IDs, emails and dates. This is more deterministic than asking an NLP model to infer structured identifiers.

```
Raw: "Rahul Sharma contacted 9876543210 and used vehicle MP04AB1234. Account 4567891234 was used for the transfer."
PERSON → Rahul Sharma
PHONE → 9876543210
VEHICLE → MP04AB1234
ACCOUNT → 4567891234
```

4.2 spaCy NER

Run a pretrained spaCy pipeline for PERSON, LOCATION and ORGANIZATION-like entities, then add project-specific EntityRuler/Matcher patterns. Return entity text, type, document ID and context.

4.3 Entity resolution

Variations such as "Rahul Sharma", "Rahul K Sharma" and "R. Sharma" should be evaluated as possible matches rather than blindly treated as separate people. Use normalization, exact matching, RapidFuzz candidate scoring, supporting attributes and a confidence threshold.

Recommended resolution pipeline

```
RAW ENTITY → NORMALIZE → EXACT MATCH → FUZZY MATCH → ATTRIBUTE OVERLAP →
CONFIDENCE → MERGE / REVIEW
```

5. PART B — DATABASE

Use PostgreSQL as the persistent source of truth. It stores canonical entities, source documents, transactions, relationships, alerts and evidence so every graph insight can be traced back to underlying records.

Core schema

Table	Important fields	Purpose
persons	id, name, normalized_name, confidence	Canonical people
phones	id, phone_number, person_id, source_id	Phone links
locations	id, name, normalized_name, latitude, longitude	Places
organizations	id, name, normalized_name	Organizations
vehicles	id, registration_number, owner_id	Vehicles/ownership
accounts	id, account_number, owner_id	Financial entities
documents	id, type, source_name, original_text, created_at	FIR/intelligence sources
transactions	id, sender_id, receiver_id, amount, timestamp, source_id	Financial flows
relationships	source_entity_id, target_entity_id, type, confidence, source_document_id, context, timestamp	Graph edges with provenance
evidence	alert_id, source_type, source_id, snippet, confidence	WHY FLAGGED support
alerts	entity_id, alert_type, severity, reason, confidence, created_at	Suspicious-pattern outputs

Relationship record design

Every relationship should answer: **who is connected to whom, by what relationship, with what confidence, and from which source/context?**

```

PERSON_001 → CONTACTED → PERSON_007
PERSON_001 → USED → PHONE_004
PERSON_001 → TRANSFERRED_MONEY → PERSON_009
PERSON_001 → LOCATED_AT → LOCATION_003

```

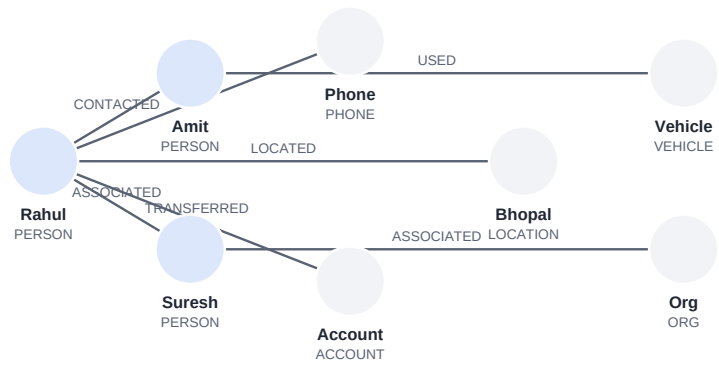
Why this matters

- PostgreSQL preserves structured relationships and source provenance.
- Foreign keys and joins support consistent cross-source queries.
- The same relationship records can be loaded into NetworkX.
- Evidence can be retrieved when an investigator opens an alert.

6. PART C — GRAPH & NETWORK

The graph layer converts database relationships into a network that exposes central actors, bridge entities, paths, groups and cycles that are difficult to see in isolated tables.

Illustrative Entity-Relationship Graph



Nodes = entities. Edges = evidence-backed relationships. Metrics rank structurally important nodes.

Graph node types

Node	Example	Typical relationships
PERSON	Rahul Sharma	CONTACTED, TRANSFERRED, ASSOCIATED_WITH
PHONE	9876543210	USED, CONTACTED
LOCATION	Bhopal	LOCATED_AT, VISITED
ACCOUNT	Account 4567	TRANSFERRED_TO, OWNED_BY
VEHICLE	MP04AB1234	USED, OWNED_BY
ORGANIZATION	Organization X	ASSOCIATED_WITH, MENTIONED_WITH
DOCUMENT	FIR_023	MENTIONS, SOURCE_OF

Network analytics

Metric / analysis	Question	Use in prototype
Degree centrality	Who has many direct connections?	Rank highly connected entities
Betweenness centrality	Who sits between separate groups?	Bridge-entity detection
Shortest path	How are two entities connected?	Investigator tracing
Connected components	Which nodes form isolated groups?	Separate network clusters
Community detection	Which nodes form dense communities?	Group-level understanding
Cycle detection	Does flow return to its origin?	Circular money-flow detection

7. Suspicious-Pattern Engine

For the MVP, use deterministic rules backed by graph/data signals. Each alert must have a reproducible condition and evidence package.

1. Communication spike

Compare communication volume against a historical or synthetic baseline over a defined time window. Example: 42 calls versus a baseline of 7/day.

```
if current_calls > historical_average × threshold:  
    create_alert(reason="COMMUNICATION_SPIKE")
```

2. Circular money flow

Detect short directed cycles in the financial graph, for example Rahul → Amit → Suresh → Rahul. Attach the relevant transaction IDs and amounts.

3. Bridge entity

Use betweenness centrality and path structure to identify nodes that connect otherwise separate communities. A bridge signal is more informative than simply counting connections.

4. Multi-source crossover

Find canonical entities or relationships that appear across multiple source types such as FIR + CDR + financial records. This directly addresses fragmented information.

Alert output

Alert	Entity	Reason	Evidence
COMMUNICATION SPIKE	Rahul	42 calls vs baseline 7/day	CDR source IDs + time window
CIRCULAR MONEY FLOW	Rahul/Amit/Suresh	Directed cycle detected	Transaction IDs + amounts
BRIDGE ENTITY	Rahul	High betweenness / community bridge	Metric + paths
MULTI-SOURCE CROSSOVER	Rahul	Appears across multiple source types	Source records + snippets

8. Evidence & Explainability (XAI)

The system should never stop at “Person X is suspicious.” It should answer “WHY FLAGGED?” with concrete, inspectable evidence.

Minimum evidence package

- **Reason code:** e.g. COMMUNICATION_SPIKE.
- **Canonical entity:** entity ID and display name.
- **Source IDs:** FIR/CDR/transaction/document identifiers.
- **Context/snippet:** the source text or relevant record fragment.
- **Confidence:** extraction or relationship confidence where applicable.
- **Graph metrics:** degree, betweenness or path information where used.
- **Rule details:** threshold, time window, cycle or path that triggered the alert.

Example WHY FLAGGED panel

WHY FLAGGED?

Entity: Rahul Sharma

Alert: MULTI-SOURCE CROSSOVER

Reason: Rahul is linked to the same canonical entity across FIR_023, CDR_441 and transaction records.

Evidence: FIR snippet mentions Rahul; CDR shows communication with Amit; transaction record shows a transfer to Amit.

Confidence: 0.94

Graph signal: high betweenness centrality.

Design constraint

The system is decision support. An alert is a signal requiring investigator review, not a determination of guilt. For the prototype, use synthetic/fictitious data.

9. Frontend & Backend

Frontend — Streamlit

Use a single investigator-oriented Streamlit dashboard. The MVP does not need a React application. Keep the interface focused on upload, analysis, search, alerts, evidence and graph exploration.

- Upload FIR text, CDR CSV and transaction CSV.
- Analyze Case button.
- Overview cards: entities, relationships, alerts, communities.
- Entity search and filters.
- Alert list with severity and reason.
- WHY FLAGGED evidence panel.
- Interactive PyVis graph.
- Shortest-path exploration.
- Source/evidence tables.

Backend options

Option	Architecture	Use when
A — Fastest MVP	Streamlit → Python services → PostgreSQL	Time is tight; recommended baseline
B — API architecture	Streamlit → FastAPI → Python services → PostgreSQL	Team is comfortable with APIs and wants separation

PyVis flow

NetworkX → PyVis → interactive HTML → Streamlit. Use node labels/types and edge relationship labels; use graph metrics to communicate structural importance.

10. Recommended Project Structure

```
criminal-network-analysis/  
├── frontend/  
│   └── streamlit_app.py  
├── backend/  
│   ├── main.py  
│   ├── config.py  
│   ├── ingestion/ (FIR, CDR, transactions)  
│   ├── extraction/ (Regex, spaCy, resolver)  
│   ├── database/ (connection, models, queries)  
│   ├── graph/ (builder, centrality, communities, paths)  
│   ├── detection/ (communication, financial, bridge, crossover)  
│   └── evidence/ (evidence_engine.py)  
├── data/ (fir, cdr, transactions)  
├── tests/  
├── requirements.txt  
├── README.md  
└── .env
```

Module responsibility

Module	Responsibility
ingestion	Read uploads and convert them into a common internal representation.
extraction	Regex + spaCy entity extraction and normalization.
database	PostgreSQL connection, schema operations and queries.
graph	NetworkX construction and graph metrics.
detection	Suspicious-pattern rules and alert creation.
evidence	Attach source/context/reason/metrics to alerts.
frontend	Dashboard, filters, tables and interactive graph.

11. Final Technology Stack

Layer	Technology	Role	Priority
Language	Python	Glue for all modules	P1
Data processing	Pandas	CSV cleaning/transformation	P1
NLP	spaCy	Natural-language entity extraction	P1
Pattern extraction	Regex / re	Structured identifiers	P1
Entity resolution	RapidFuzz	Duplicate/similar matching	P1
Database	PostgreSQL	Persistent source of truth	P1
DB driver	psycopg	Python ↔ PostgreSQL	P1
Graph	NetworkX	Graph construction/analysis	P1
Visualization	PyVis	Interactive network	P1
Detection	Python rules	Explainable suspicious patterns	P1
Frontend	Streamlit	Investigator dashboard	P1/P2
Backend API	FastAPI	Optional API boundary	P2/P3
ML	scikit-learn	Optional learned anomaly layer	P3
Collaboration	Git + GitHub	Version control/teamwork	P1

Priority key: P1 = critical to the demo; P2 = supporting; P3 = optional after the MVP is reliable.

12. Scope Control — What NOT to Build First

Do not overengineer the first prototype. These technologies should not be prerequisites for the MVP:

- Neo4j as a mandatory database.
- React + Cytoscape frontend.
- BERT/transformer-heavy NLP.
- Graph Neural Networks.
- Deep anomaly detection.
- Multilingual NLP before the base pipeline works.
- Enterprise authentication/authorization.
- Microservices/Kubernetes.
- Cloud deployment before local reliability.
- A sophisticated ML classifier with no explainable reason.

Future enhancement path

After the core loop is stable, advanced components can be evaluated for scale and accuracy: dedicated graph databases, transformer NLP, learned anomaly detection, GNNs, multilingual processing, stronger access control and production deployment.

Do not present future technologies as implemented features unless they actually exist in the prototype.

13. Final Demo Acceptance Checklist

1. Synthetic FIR text uploads successfully.
2. CDR CSV uploads successfully.
3. Financial transaction CSV uploads successfully.
4. Regex extracts structured identifiers.
5. spaCy extracts text entities.
6. Entity resolution links/merges some duplicate records.
7. PostgreSQL stores canonical entities and relationships.
8. Relationships retain source/context and confidence.
9. NetworkX graph is generated.
10. Degree/betweenness or another meaningful metric is shown.
11. Communication spike alert works.
12. Circular money-flow alert works.
13. Bridge-entity alert works.
14. Multi-source crossover alert works.
15. Every alert has a WHY FLAGGED explanation.
16. Evidence points back to source records/snippets.
17. PyVis graph is interactive.
18. Streamlit runs the complete flow.
19. The team can explain what each node, edge, metric and alert means.
20. Demo uses synthetic/fictitious data and clearly states decision-support scope.

Single most important acceptance criterion

Raw fragmented data → entities → resolved entities → relationships → graph → suspicious pattern → evidence → investigator dashboard.

14. Team Learning Priorities

Divide ownership, but require every member to understand the end-to-end pipeline well enough to debug and explain it.

Area	Primary technologies	Owner must understand
Database + Graph	PostgreSQL, psycopg, Pandas, RapidFuzz, NetworkX	Schema, entity resolution, graph construction, centrality, paths, alerts and evidence flow.
AI / NLP	spaCy, Regex, EntityRuler/Matcher	Entity extraction, structured patterns, errors and output contract.
Frontend / Backend	Streamlit, PyVis, Python, FastAPI if used	Upload → process → query → display; integration and UI state.

Debugging rule

For every module, the developer should know what data enters the function, what comes out, which database table or graph structure is touched, and what condition creates an alert. AI-generated boilerplate can accelerate implementation, but the team must be able to explain and test it.

Final architecture statement

The prototype should be evaluated on whether it makes fragmented information easier to connect, analyze and explain. A smaller stack that reliably demonstrates the complete evidence-backed pipeline is stronger than a large stack of unused technologies.